Part 4: Squashing the Decryption Circuit and Bootstrapping

Speaker Notes & Mathematical Primer

Nirav Bhattad

March 27, 2026

§1. The Bootstrapping Blueprint

How to start speaking: "We have a severe problem. Our Somewhat Homomorphic Encryption (SHE) scheme can perform additions and multiplications, but every multiplication squares the noise. Eventually, the noise exceeds $p/2$, and the ciphertext is permanently destroyed. How do we evaluate a circuit of arbitrary depth?
The answer lies in Craig Gentry's brilliant insight from 2009: **Bootstrapping**. If an encryption scheme is capable of homomorphically evaluating its *own* decryption circuit, plus one additional gate (like a NAND gate), it can evaluate any circuit of any depth."

The bootstrapping theorem works by taking a "noisy" ciphertext c (where the noise is perilously close to $p/2$) and encrypting it again under a new public key. We also provide an encrypted version of the secret key. The scheme then homomorphically evaluates the decryption algorithm on the inner ciphertext. Because we are evaluating the decryption circuit *homomorphically*, the output is not the plaintext, but a fresh ciphertext encrypting the same message! This new ciphertext has a fixed, smaller amount of noise determined only by the depth of the decryption circuit, effectively "refreshing" the ciphertext.

Definition 1.1: Bootstrappable Encryption

Let $\mathcal{E}$ be a homomorphic encryption scheme, and let $\mathcal{C}_{\mathcal{E}}$ be the set of permitted circuits it can evaluate before noise exceeds the bounds. Let $D_{\mathcal{E}}$ be the set of augmented decryption circuits (decryption plus one gate). We say $\mathcal{E}$ is bootstrappable if $D_{\mathcal{E}} \subseteq \mathcal{C}_{\mathcal{E}}$.

§2. The Depth Problem

The Roadblock: "So, we just need to evaluate our decryption circuit. Let's look at our decryption equation: $m' = (c \bmod p) \bmod 2$.
Can our SHE scheme handle this? Mathematically, $c \bmod p = c - p\lfloor c/p \rfloor$. To evaluate this homomorphically, we must compute $\lfloor c/p \rfloor$. This requires integer division, which, when translated into a boolean polynomial circuit, has a massive multiplicative depth.
Our permitted circuits in the SHE scheme can only handle polynomials up to degree $d \leqslant \frac{\eta - 4 - \log|\vec{f}|}{\rho' + 2}$.

The standard division circuit's degree far exceeds this bound. The decryption circuit is simply too deep. We must 'squash' it."

§3. Squashing the Decryption Circuit

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit. We do this by shifting the heavy computational lifting from the Decrypt algorithm (which is constrained) to the Evaluate algorithm (which can be done in the clear by the server before the final decryption step).
We introduce three new parameters: κ (precision), Θ (size of a public set), and θ (the Hamming weight of a sparse subset).

3.1. Modifying the Key Generation

During KeyGen, alongside the secret key p and the public key pk^*, we generate a set of rational numbers.

1. Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. The vector $\vec{s}$ becomes the new secret key.
2. Generate a set $\vec{y} = \{y_1, \ldots, y_\Theta\}$ of rational numbers in $[0, 2)$ with κ bits of precision, such that:

$$\sum_{i \in S} y_i \approx \frac{1}{p} \pmod 2 \tag{3.1}$$

   Specifically, $\left[\sum_{i \in S} y_i\right]_2 = \frac{1}{p} - \Delta_p$, where $|\Delta_p| < 2^{-\kappa}$.

The public key is now $pk = (pk^*, \vec{y})$.

The Intuition of Squashing: "We have hidden a sparse subset sum within the public key that approximates $1/p$. Why? Because if the server knows an approximation of $1/p$, it can multiply the ciphertext by $1/p$ *for us*, saving the decryption circuit from doing the division!"

3.2. Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. Before returning it, the server does some post-processing: For $i \in \{1, \ldots, \Theta\}$, the server computes:

$$z_i = [c^* \cdot y_i]_2 \tag{3.2}$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision after the binary point. The server outputs c^* and the vector $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.
Now, let us derive the new, "squashed" decryption circuit. Recall that $c/p \bmod 2$ is what we need to compute.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2 \tag{3.3}$$

$$= \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \tag{3.4}$$

$$\approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2 \tag{3.5}$$

Therefore, the modified Decrypt algorithm simply computes:

$$m' = \left[c^* - \left\lfloor \sum_{i=1}^{\Theta} s_i z_i \right\rfloor \right] \bmod 2 \tag{3.6}$$

§4. The Shallow Decryption Circuit

The Climax of the Proof: "Look at our new decryption equation. The server has already given us the rational numbers z_i. The secret key bits s_i are just 0 or 1. Our only job is to sum up Θ numbers, but because $\vec{s}$ is sparse, only θ of them are non-zero!
Can our SHE scheme evaluate the sum of θ rational numbers? Yes! We use the classic 'three-for-two' trick to reduce the sum, and we compute the carry bits using elementary symmetric polynomials."

Lemma 4.1

The i-th bit of the Hamming weight of a binary vector $\vec{\sigma}$ can be expressed as a polynomial of degree exactly 2^i using the 2^i-th elementary symmetric polynomial $e_{2^i}(\vec{\sigma})$.

Because we only need $\lceil \log \theta \rceil + 3$ bits of precision, the maximum degree of the polynomials required to compute the binary sum and rounding is bounded by:

$$\text{Degree} \leqslant 64\theta \log^2 \theta \tag{4.1}$$

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial is $\mathcal{O}(\lambda \log^2 \lambda)$.
If we look back at our permitted polynomials for the SHE scheme (from Part 2), we see that by setting $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the scheme can comfortably handle polynomials of this degree.

Theorem 4.2: Bootstrapping Achieved

Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials ($\mathcal{C}_{\mathcal{E}}$), we have $D_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$. Therefore, the scheme is bootstrappable and, consequently, Fully Homomorphic.

Wrapping up Part 4: "And there we have it. By publishing an approximation of $1/p$ and forcing the evaluator to pre-compute the partial products z_i, we squashed the decryption circuit down to a simple sparse subset sum. This sum has a low enough polynomial degree to be evaluated homomorphically by the base scheme itself. We have achieved Fully Homomorphic Encryption using nothing but modular arithmetic over the integers."